



Wat is de huidige gouden standaard voor privacy-first architectuur

in mental health en recovery apps die hooggevoelige persoonlijke data verwerken? Bespreek: zero-knowledge encryptie implementaties (Web Crypto API, Signal Protocol), local-first dataopslag (IndexedDB, CRDTs), anonieme authenticatiemethoden, GDPR Artikel 9 vereisten voor gezondheidsdata, hoe apps als Standard Notes en Proton end-to-end encryptie implementeren, en wat 'privacy by design' in de praktijk betekent voor een traumaherstelapp. Wat zijn de aansprakelijkheidsimplicaties bij versleutelde peer-communicatie?

De huidige gouden standaard is een combinatie van zero-knowledge end-to-end encryptie, local-first opslag, minimale metadata, en strikte juridische scheiding tussen “wellness/recovery support” en gereguleerde zorgfunctionaliteit. Voor een mental health of traumaherstelapp betekent dat in de praktijk: data standaard lokaal versleuteld bewaren, sleutels uitsluitend op het device afleiden of genereren, de server reduceren tot een “dumb relay/store”, en alleen Artikel 9-gezondheidsdata verwerken als je daar een expliciete en aantoonbare rechtsgrond plus passende waarborgen voor hebt.^{[1] [2] [3] [4]}

Architectuur

Een privacy-first referentie-architectuur begint met client-side encryptie via de Web Crypto API voor sleutelafleiding, sleutelbeheer en AEAD-encryptie in de browser, terwijl voor versleutelde peer-communicatie meestal een ratcheting protocol in de Signal-familie wordt gebruikt in plaats van zelf iets op maat te ontwerpen. De Web Crypto API levert de cryptografische primitieven voor webapps, terwijl Signal's X3DH/PQXDH plus Double Ratchet forward secrecy, break-in recovery en per-bericht unieke sleutels biedt voor asynchrone communicatie.^{[5] [6] [4]}

Voor opslag is local-first nu de beste praktijk: gevoelige notities, dagboeken, triggers, herstelplannen en conceptberichten eerst lokaal opslaan, bijvoorbeeld in IndexedDB, en pas daarna versleuteld synchroniseren. CRDTs zijn zinvol als je offline-first samenwerking of multi-device conflictresolutie wilt, maar ze zijn geen verplichting; ze zijn vooral nuttig bij echte collaboratie of complexe synchronisatie, niet bij elke individuele journaling-app.^{[7] [8] [1]}

Praktisch patroon voor een traumaherstelapp:

- Lokale datastore in IndexedDB, met records versleuteld per item of per collectie.^{[6] [8]}

- Master key afgeleid uit passphrase of device-bound secret, nooit in plaintext naar de server.^{[9] [1]}
- Random data keys voor individuele items of domeinen zoals journal, safety plan, messages.^[1]
- Sync-server die alleen ciphertext, versleutelde item keys en minimale routingmetadata ziet.^{[3] [1]}

Zero-knowledge

Standard Notes is een sterk voorbeeld van zero-knowledge ontwerp: het beschrijft expliciet een model waarin de server als “dumb data-store” wordt behandeld en geen geheime waarden hoeft te kennen. Het systeem splitst een uit een wachtwoord afgeleide root key in een lokale master key en een aparte server password-component; syncbare data wordt versleuteld met willekeurige items keys die zelf weer met de master key worden versleuteld.^[1]

Hun specificatie noemt Argon2id voor key derivation en XChaCha20-Poly1305 voor encryptie, en zegt ook dat lokale opslag client-side wordt versleuteld en dat de master key waar mogelijk in een secure keychain zit of anders via een extra wrapper met lokale passcode wordt beschermd. Dat is relevant voor recovery-apps, omdat je daarmee zowel “cloud blind” opslag als apparaatbescherming combineert zonder dat de server inhoud kan ontsleutelen.^{[9] [1]}

Proton gebruikt een vergelijkbaar zero-access model voor opgeslagen data: berichten en bijlagen in Proton Mail worden met zero-access encryptie opgeslagen, en berichten tussen Proton-gebruikers zijn end-to-end encrypted. Proton benadrukt tegelijk een belangrijke realiteit voor architectuurkeuzes: sommige metadata en interoperabiliteitsvelden, zoals e-mailheaders of subject lines, zijn niet volledig end-to-end encrypted door protocolbeperkingen, dus “inhoud versleuteld” is niet hetzelfde als “alles verborgen”.^[3]

Voor een mental health app is de les dus: claim geen absolute anonimiteit als je nog steeds metadata verwerkt zoals contactgrafieën, push tokens, IP-adressen, crisis-escalatie-events of tijdstempels. De gouden standaard is niet alleen sterke inhoudsversleuteling, maar ook metadata-minimalisatie, scheiding van identifiers en optionele bescherming van headers of routinginformatie waar technisch haalbaar.^{[10] [5] [3]}

Authenticatie

Anonieme of pseudonieme authenticatie is hier vaak beter dan klassieke accountmodellen met naam, e-mail en telefoonnummer als standaard. Een privacy-first app kan werken met willekeurige account-ID's, passkeys, device-bound keys, recovery codes en optionele pseudonieme login in plaats van verplichte echte identiteit.^{[3] [1]}

Goede patronen zijn:

- Pseudoniem account in plaats van echte naam.^[3]
- Passkey of lokaal sleutelbaar als primaire authenticatie.^[6]

- E-mail alleen optioneel voor recovery of notificaties, logisch gescheiden van gezondheidsdata.^[2]
- Blind of minimal token-based auth voor sync, zodat authenticatiegegevens en inhoudelijke data niet in één dataset samenkomen.^[1]

Wat je liever vermijdt in een traumaherstelapp is verplichte sociale login, verplichte telefoonnummerverificatie of gecentraliseerde profielverrijking. Zulke patronen vergroten heridentificatierisico, maken DPIA-uitkomsten zwaarder en botsen vaak met het beginsel van dataminimalisatie.^{[11] [2]}

GDPR Artikel 9

Gezondheidsdata valt onder de bijzondere categorieën van persoonsgegevens, en Artikel 9 lid 1 verbiedt verwerking in beginsel tenzij een uitzondering uit lid 2 geldt. Expliciete toestemming is één mogelijke grondslag, maar niet de enige; voor health of social care kunnen ook andere gronden relevant zijn, mits wettelijk verankerd en voorzien van passende safeguards.^[2]

Voor een commerciële traumaherstelapp zonder formele behandelrelatie is expliciete toestemming vaak de meest voor de hand liggende Artikel 9-grondslag, maar die moet echt expliciet, specifiek en aantoonbaar zijn, en staat los van de gewone Artikel 6-grondslag. Encryptie helpt sterk als passende maatregel, maar vervangt de rechtsgrond niet; ook pseudonimisering of end-to-end encryptie maakt gezondheidsdata niet automatisch “geen persoonsgegevens” zolang iemand redelijkerwijs herleidbaar blijft.^{[11] [2]}

In de praktijk betekent privacy by design onder GDPR hier:

- Alleen data verzamelen die nodig is voor het hersteldoel.^[2]
- Gezondheidsvelden opt-in maken en standaard uit laten.^[2]
- Lokale verwerking als default, cloudsync optioneel.^[1]
- DPIA uitvoeren vóór livegang, gezien de aard en gevoeligheid van de data.^[2]
- Heldere rolafbakening: ben je wellness tool, peer-support platform of zorgaanbieder? Die keuze bepaalt bewaartermijnen, incidentrespons en governance.^{[11] [2]}

Peer-communicatie

Bij versleutelde peer-communicatie verschuift aansprakelijkheid, maar verdwijnt die niet. Als je Signal-achtige end-to-end encryptie gebruikt, kun je meestal aannemelijk maken dat je geen toegang hebt tot inhoud van berichten, omdat Signal's documentatie laat zien dat de inhoud met per-bericht sleutels, ratchets en forward secrecy wordt beschermd.^{[4] [5]}

Dat beperkt je exposure voor inhoudelijke inzage, maar niet voor andere risico's:

- Je blijft verantwoordelijk voor metadata die je wel verwerkt, zoals accountkoppelingen, tijdstippen, abuse-signalen en IP-gerelateerde logs.^{[10] [3]}
- Je blijft verantwoordelijk voor productontwerpkeuzes, zoals matchmaking, rapportageflows, crisiswaarschuwingen en moderation UX.^[2]

- Je kunt nog steeds aansprakelijkheidsrisico lopen als je onveilige claims maakt, bijvoorbeeld “volledig anoniem” of “geschikt voor crisiszorg”, terwijl de architectuur dat niet waarmaakt.^{[10] [2]}

Voor een traumaherstellapp is vooral belangrijk dat end-to-end encryptie moderatie en duty-of-care ingewikkelder maakt. Als je inhoud niet kunt lezen, kun je ook niet vertrouwen op server-side detectie van suïcidaliteit, grooming, dwang of escalatie; dan moet je aansprakelijkheidsmatig compenseren met productwaarborgen zoals client-side safety nudges, user-controlled reporting, block/mute, consent gates, crisis disclaimers en duidelijke escalatiepaden buiten de versleutelde inhoud om.^{[4] [10] [2]}

Een bruikbare juridische vuistregel is daarom: end-to-end encryptie verlaagt je aansprakelijkheid voor ongeautoriseerde inhoudstoegang, maar verhoogt je verantwoordelijkheid om vooraf aantoonbaar veilige governance, risicoanalyse en verwachtingsmanagement in te bouwen. Voor een Europese recovery-app is de beste positie meestal: zero-knowledge waar mogelijk, minimale metadata, geen onnodige centrale inzage, expliciete Artikel 9-grondslag, en een duidelijk afgebakend product dat geen klinische interventies suggereert zonder passende regulatoire basis.^{[3] [1] [2]}

Wil je dat ik dit omzet naar een concrete referentie-architectuur voor een Next.js webapp, inclusief datamodel, key management flow en DPIA-checklist?



1. <https://standardnotes.com/help/security>
2. <https://gdpr-info.eu/art-9-gdpr/>
3. <https://proton.me/support/proton-mail-encryption-explained>
4. <https://signal.org/docs/>
5. <https://standardnotes.com/help/security/encryption>
6. https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API
7. <https://github.com/alexanderop/awesome-local-first/blob/main/README.md>
8. <https://joodaloop.com/local-first/>
9. <https://standardnotes.com/help/3/how-does-standard-notes-secure-my-notes>
10. <https://standardnotes.com/knowledge/2/what-is-end-to-end-encryption>
11. <https://sfc.ec.europa.eu/en/can-we-collect-personal-data-health-related-interventions-without-breaking-eu-rules-data-protection>
12. <https://spin.atomicobject.com/standard-notes-review/>
13. https://www.reddit.com/r/crypto/comments/1gmcwk1/webapp_encryption_at_rest/
14. <https://standardnotes.com>
15. <https://inria.hal.science/hal-02294935v1/document>
16. <https://apps.apple.com/nl/app/standard-notes/id1285392450?l=en-GB>
17. https://www.reddit.com/r/ProtonMail/comments/vqszmm/privacy_benefits_outside_of_endtoend_encryption/

18. <https://github.com/standardnotes/snjs/blob/main/packages/snjs/specification.md>
19. <https://gdpr.eu/what-is-gdpr/>
20. <https://proton.me/support/how-to-use-pgp>
21. <https://proton.me/blog/encrypted-email>
22. <https://proton.me/blog/what-is-pgp-encryption>
23. <https://proton.me/support/mail/email-encryption>
24. https://en.wikipedia.org/wiki/Signal_Protocol
25. <https://github.com/ProtonMail/gopenpgp>
26. <https://proton.me/mail/security>
27. [https://csrc.nist.gov/csrc/media/events/2025/sixth-pqc-standardization-conference/a comprehensive study of the signal handshake protocol \(2\).pdf](https://csrc.nist.gov/csrc/media/events/2025/sixth-pqc-standardization-conference/a%20comprehensive%20study%20of%20the%20signal%20handshake%20protocol%20(2).pdf)
28. <https://rxdb.info/articles/local-first-future.html>
29. <https://longguy.com/how-proton-mail-uses-end-to-end-encryption-the-architecture-behind-private-email>
30. <https://www.signal.org/docs/>